

**«Санкт-Петербургский государственный электротехнический
университет «ЛЭТИ» им. В. И. Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: UI-тестирование

Студенты гр. 4388

Арефьев И. Д.
Денисенко В. С.
Донцова И. Е.
Шарапов Д. Р.

Руководитель

Шевелева А. М.

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студентка: Донцова И. Е.

Группа: 4388

Тема практики: UI-тестирование

Задание на практику:

Задание на практику заключалось в разработке автоматизированных тестов для проверки пользовательского интерфейса (UI-тестирования) веб-сайта Rutube. Основная работа включала изучение основ автоматизации тестирования на языке Java с использованием инструментов Selenide, а также проектирование архитектуры проекта на основе Page Object Model.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студентка

Донцова И. Е.

Руководитель

Шевелева А. М.

АННОТАЦИЯ

В данном отчёте представлена работа по разработке набора автоматизированных UI-тестов для веб-приложения RuTube. Проект реализован на языке Java с применением фреймворков Selenide [1], JUnit 5 [2] и AssertJ [3], а также системы сборки Maven [4].

Архитектура проекта построена на основе паттерна Page Object Model [5], что обеспечивает разделение логики взаимодействия со страницами и логики проверок, упрощая поддержку и расширение тестов. В проекте реализована автоматическая авторизация через загрузку сессионных cookies и обработка всплывающих окон.

В рамках практики разработано 10 автоматизированных тестов, покрывающих основные пользовательские сценарии: поиск и фильтрацию видео, управление воспроизведением, настройку качества, оценку контента, подписки на каналы и работу с плейлистами. Все тесты успешно выполняются, что подтверждает корректность работы проверяемого функционала.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1. ПОСТАНОВКА ЗАДАЧИ.....	8
1.1 Предметная область	8
1.2 Задачи практики	8
2. РЕЗУЛЬТАТЫ РАБОТЫ	9
2.1. Описание реализуемых тестов.....	9
2.1.1 Проверка поиска.....	9
2.1.2 Проверка работы с каналами.....	11
2.1.3 Проверка взаимодействия с видео	12
2.1.4 Результат выполнения всех тестов.....	18
2.2 Описание классов и методов	19
2.2.1. Блок базовых классов (base)	19
2.2.2 Блок элементов интерфейса (elements)	20
2.2.3 Блок страниц (pages)	22
2.2.4 Блок тестов (tests).....	25
2.2.5 Блок утилит (utils)	27
2.2.6 Диаграмма классов (UML).....	27
3 ИНДИВИДУАЛЬНАЯ РАБОТА	30
3.1 Создание структуры проекта	30
3.2 Настройка файла сборки pom.xml	30
3.3 Создание иерархии базовых классов.....	31
3.4 Работа над документацией.....	31
3.5 Сортировка методов в классах	32
3.6 Результаты выполненной работы	32
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	33
4.1 Язык программирования	33
4.2 Система сборки.....	33
4.3 Фреймворки для тестирования	33
4.4 Инструменты для управления браузером	34
4.5 Вспомогательные библиотеки	34

4.6 Инструменты разработки	34
4.7 Используемые подходы и паттерны	34
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	36
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39

ВВЕДЕНИЕ

Предметной областью данной практики является автоматизация тестирования для обеспечения качественной работы веб-приложений, таких как Rutube. Автоматизированное UI-тестирование позволяет имитировать действия реальных пользователей, находить ошибки при обновлении дизайна или логики сайта, а также значительно ускоряет процесс проверки работоспособности.

Целью учебной практики является создание 10 автоматизированных UI-тестов для проверки ключевых функций и интерфейса Rutube с использованием Java, Selenide, JUnit 5.

Задачи учебной практики:

1. Изучение автоматизации с использованием языка Java и библиотеки Selenide.
2. Проектирование структуры проекта по принципу Page Object Model для разделения логики проверок и элементов интерфейса
3. Программное описание основных страниц и компонентов сайта
4. Настройка автоматической авторизации через JSON-файлы с куками
5. Написание 10 автотестов для проверки различных сценариев использования платформы.

Поставленное задание заключалось в командной разработке автотестов для проверки интерфейса Rutube. Требовалось спроектировать удобную структуру папок, подключить нужные библиотеки, написать 10 автотестов для ключевых функций и настроить автоматический вход в аккаунт. Также важной частью задания была совместная отладка кода и исправление всех ошибок по замечаниям преподавателя.

Вклад каждого участника команды:

- Арефьев И. Д. (тимлид): создал общий репозиторий на GitHub [6], написал поясняющие комментарии в коде для разных функций, а также исправил ошибки в нескольких тестах, чтобы они запускались без сбоев.

- Денисенко В. С.: составила чек-лист для проверок, настроила автоматическую авторизацию через чтение сессионных кук из JSON-файла, исправила несколько тестов, чтобы они корректно работали, и внесла изменения по требованиям преподавателя.
- Донцова И. Е.: написала основной каркас для всего проекта, создала структуру папок по шаблону Page Object Model и настроила файл сборки проекта `pom.xml`, а также внесла изменения в код по замечаниям преподавателя.
- Шарапов Д. Р.: занимался доработкой и чисткой кода, исправил логику работы видеоплеера и поисковых фильтров, добавил валидацию запросов, а также внес исправления по замечаниям преподавателя.

1. ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Предметной областью данной практики является автоматизация тестирования пользовательского интерфейса для обеспечения качественной работы веб-приложений, таких как Rutube. Автоматизированное UI-тестирование позволяет имитировать действия реальных пользователей. Программа сама открывает браузер, вводит текст, кликает по элементам и проверяет, правильно ли отработал сайт. Автотесты помогают быстрее находить ошибки после изменения дизайна или логики сайта, а также значительно ускоряет процесс проверки работоспособности.

1.2 Задачи практики

В рамках прохождения учебной практики были поставлены следующие задачи:

1. Изучение автоматизации с использованием языка Java и библиотеки Selenide.
2. Проектирование структуры проекта по принципу Page Object Model для разделения логики проверок и элементов интерфейса
3. Программное описание основных страниц и компонентов сайта
4. Настройка автоматической авторизации через JSON-файлы с куками
5. Написание 10 автотестов для проверки различных сценариев использования платформы.

2. РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Описание реализуемых тестов

В этом разделе описывается работа 10 написанных автотестов для Rutube. Для удобства все тесты поделены на три группы: поиск, работа с каналами и проверка самого видеоплеера.

2.1.1 Проверка поиска

В этой группе тестов проверяется, как сайт ищет видео. Здесь проверяется, правильно ли обновляются результаты, если поменять слово в строке поиска, не ломается ли страница от пустого запроса, и корректно ли работают фильтры по дате добавления и длительности ролика.

Тест 2. Применение фильтров: Тест вводит запрос «новости», нажимает на кнопку «Найти» (рис. 1), открывает панель фильтров и выбирает значения «За сегодня» и «20–60 минут» (рис. 2). Затем он берет первое видео из списка и проверяет, что оно действительно опубликовано сегодня и его длительность попадает в этот диапазон.



Рисунок 1 — Ввод поискового запроса «Новости»

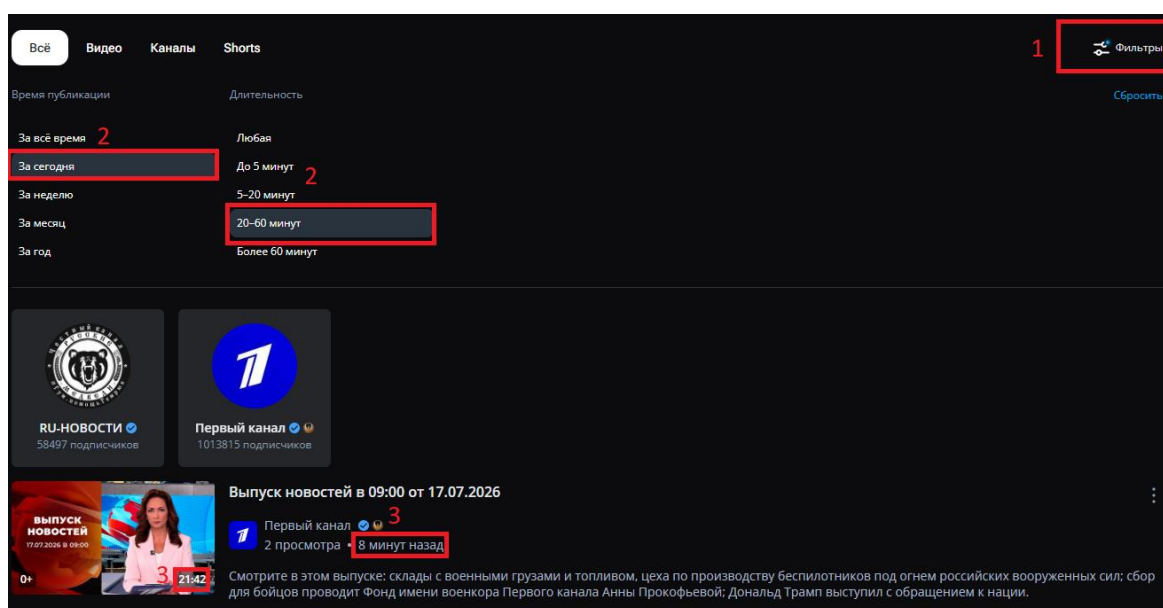


Рисунок 2 — Применение фильтров и проверка видео на соответствие

Тест 3. Изменение запроса: Тест ищет видео по слову «музыка» и запоминает название первого ролика (рис. 3). После этого он очищает поле ввода с помощью крестика, вводит запрос «кино» и проверяет, что поле поиска обновилось, а результаты на странице изменились (рис. 4).

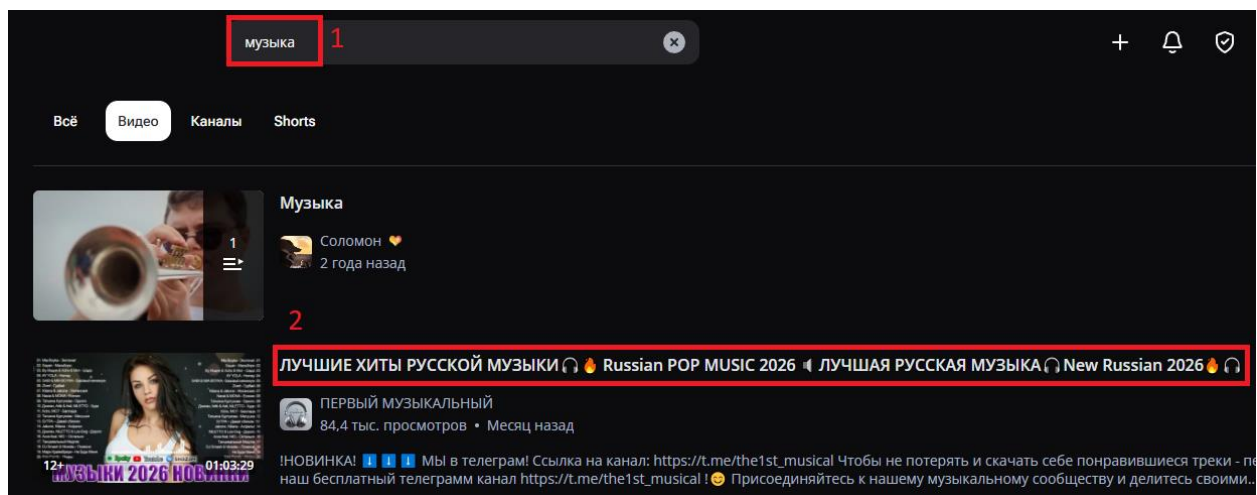


Рисунок 3 — Ввод поискового запроса «Музыка» и запоминание названия видео

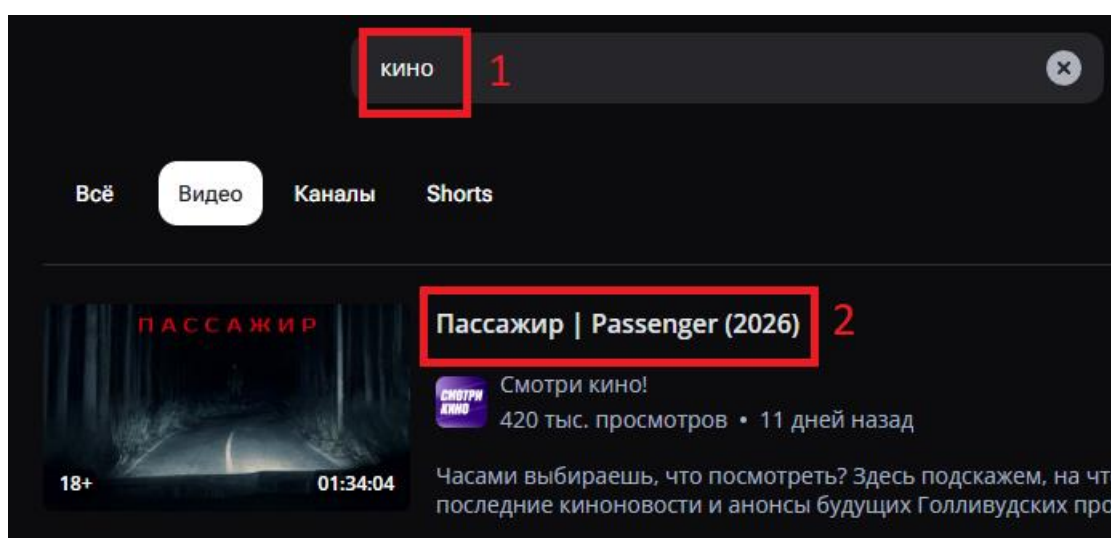


Рисунок 4 — Ввод поискового запроса «Кино» и проверка названия видео

Тест 5. Поиск с пустым запросом: Проверяет реакцию сайта на пустую строку поиска. Тест отправляет пустой запрос, через левое меню переходит в раздел «В топе» (рис. 5), а затем кликает на логотип, чтобы вернуться на главную страницу (рис. 6). Проверяется, что интерфейс не сломался и главная страница открылась.

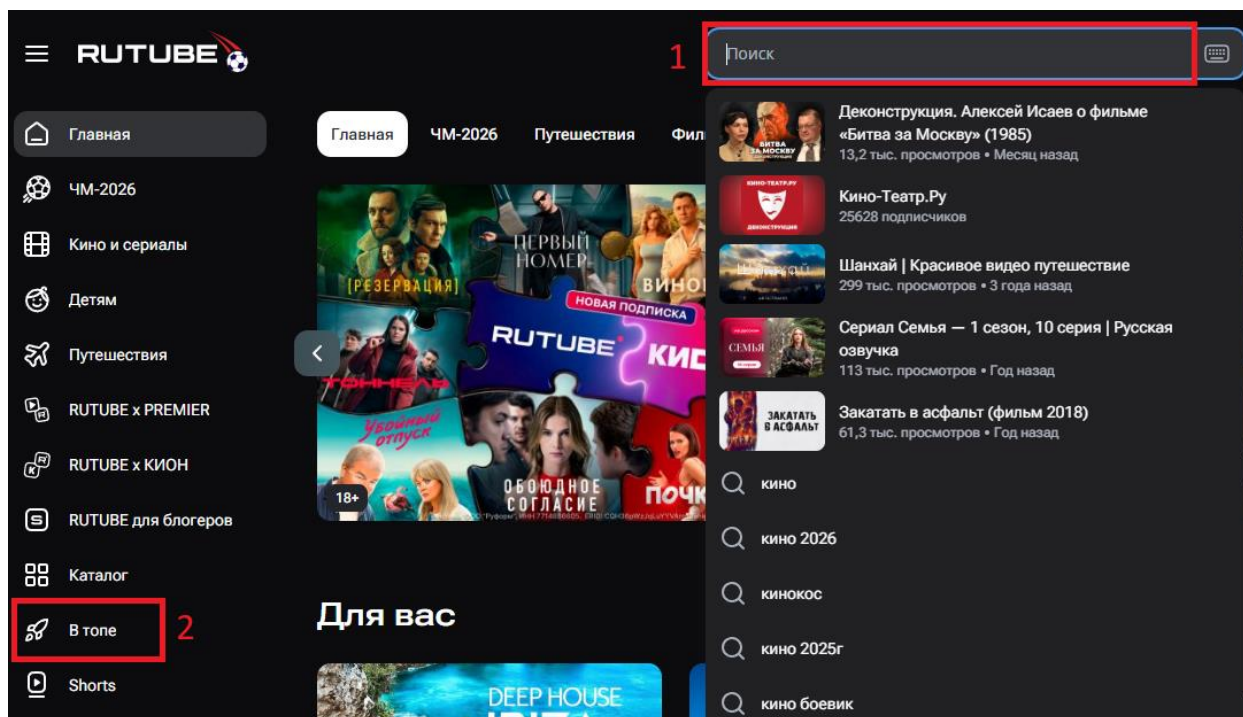


Рисунок 5 — Ввод пустого поискового запроса и переход в раздел «В топе»

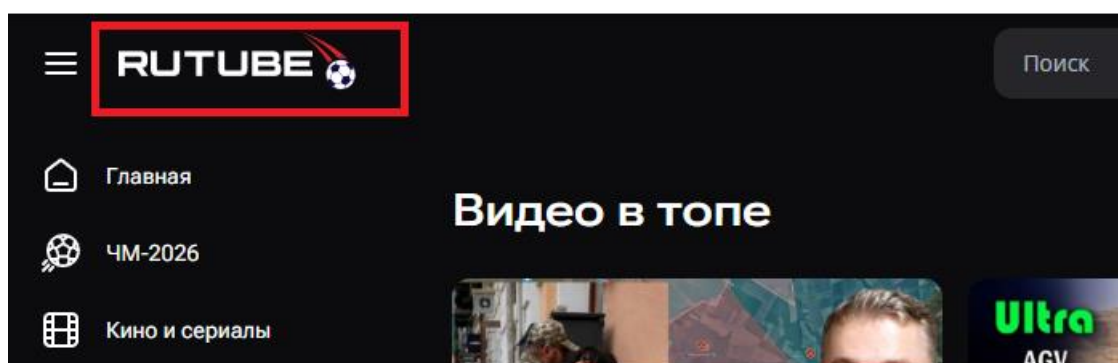


Рисунок 6 — Переход на главную страницу из раздела «В топе»

2.1.2 Проверка работы с каналами

Здесь тест, который проверяет, что можно найти нужный канал, подписаться на него и проверить статус подписки.

Тест 6. Поиск канала и оформление подписки: Тест вводит запрос «Матч ТВ», переключается на вкладку «Каналы» и нажимает кнопку «Подписаться» у первого найденного канала (рис. 7). Затем он переходит в профиль этого канала и проверяет, что кнопка поменяла статус на «Вы подписаны» (рис. 8).

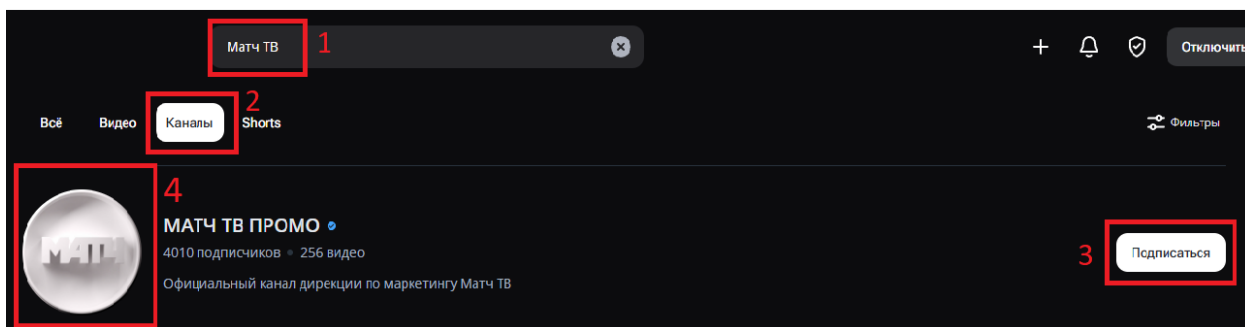


Рисунок 7 — Ввод поискового запроса «Матч ТВ», переключение на вкладку «Каналы», клик по кнопке «Подписаться» и переход в профиль канала



Рисунок 8 — Проверка статуса кнопки в профиле канала

2.1.3 Проверка взаимодействия с видео

Это самая большая часть тестов, в которой проверяется всё, что связано с просмотром. Сюда входит проверка самого плеера (пауза, смена качества, полноэкранный режим), а также возможность ставить лайки и дизлайки, сохранять ролики в плейлист «Смотреть позже», копировать ссылки и отправлять жалобы на видео.

Тест 1. Поиск видео: Тест ищет ролики по слову «музыка», открывает первое видео (рис. 9) и нажимает на кнопку паузы (рис. 10) в открывшемся плеере. Проверяется, что воспроизведение успешно приостановилось.

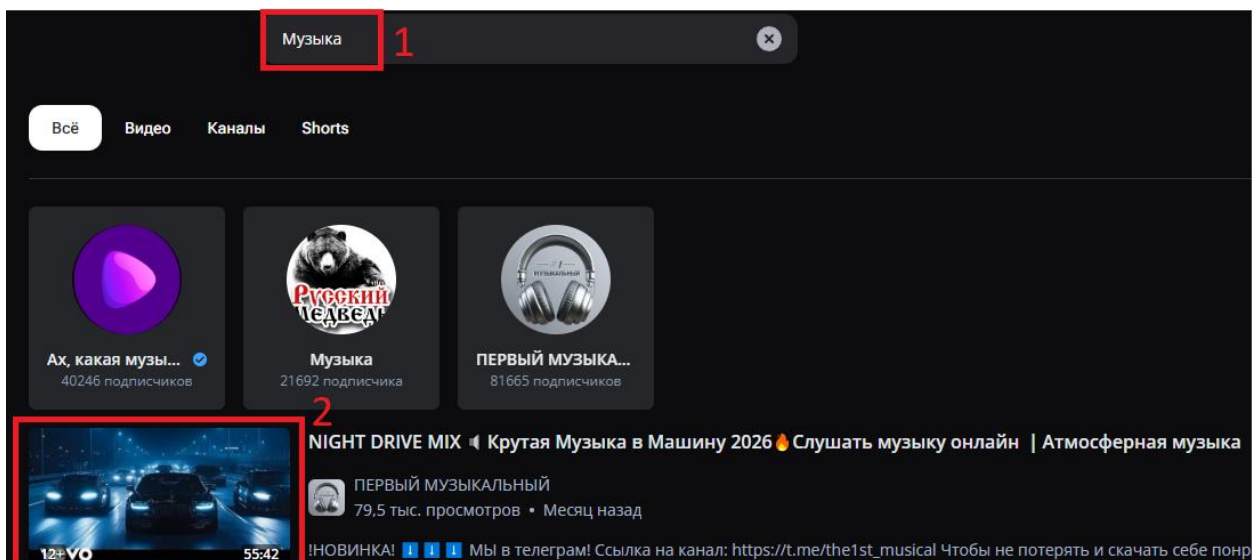


Рисунок 9 — Ввод поискового запроса «Музыка» и открытие первого видео

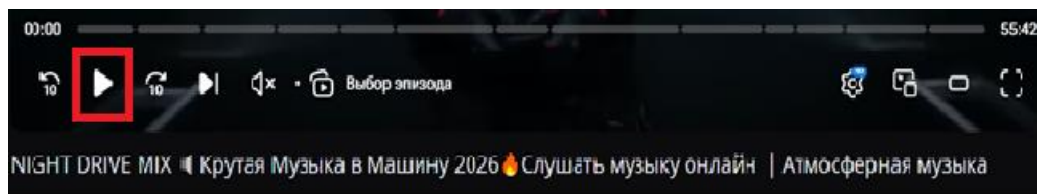


Рисунок 10 — Наведение на плеер и нажатие кнопки паузы

Тест 4. Отправка жалобы на видео: Тест ищет «фильмы», открывает первое видео и нажимает кнопку «Пожаловаться» (рис. 11 и рис. 12). Проверяется, что форма жалобы успешно открылась на экране, после чего тест закрывает эту форму (рис. 13).

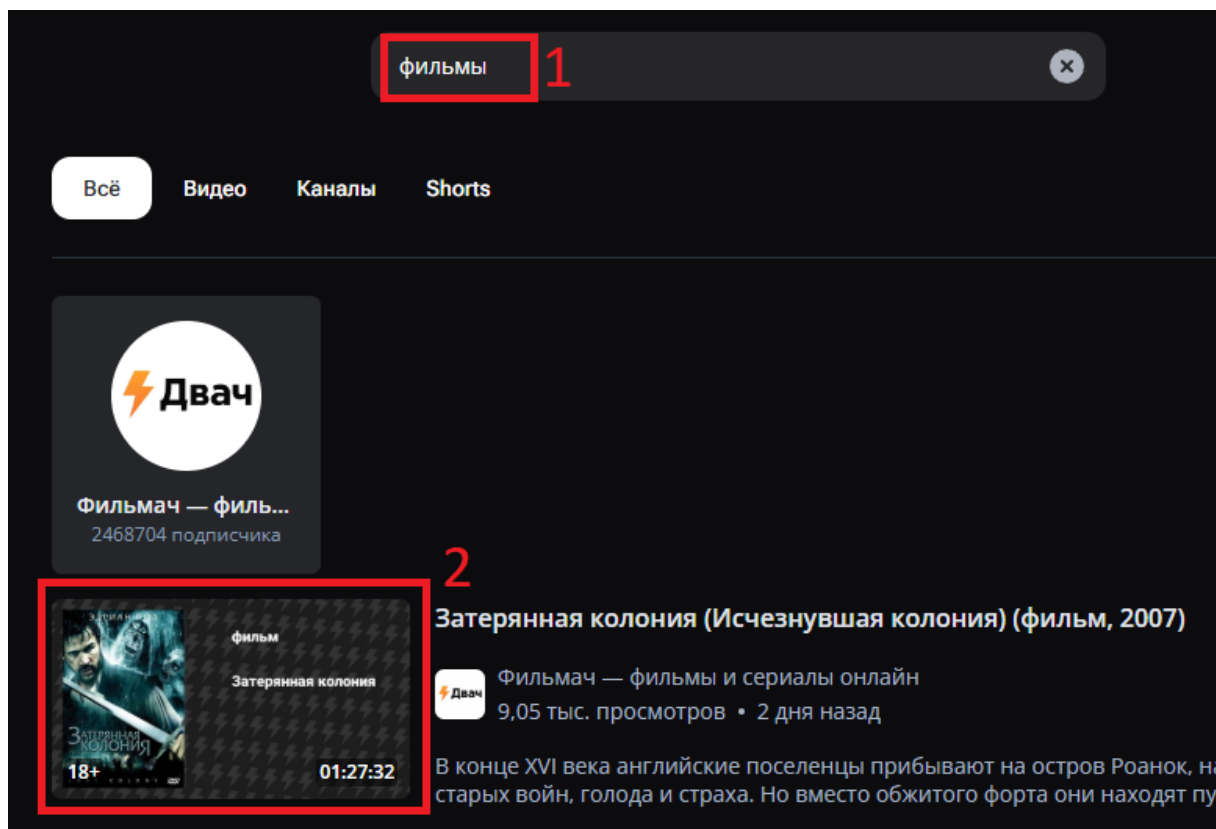


Рисунок 11 — Ввод поискового запроса «Фильмы» и открытие первого видео

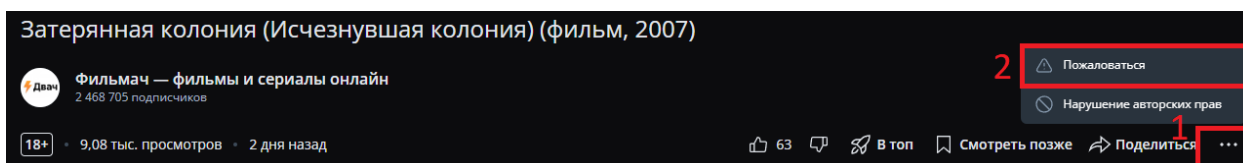


Рисунок 12 — Открытие контекстного меню и клик по кнопке
«Пожаловаться»

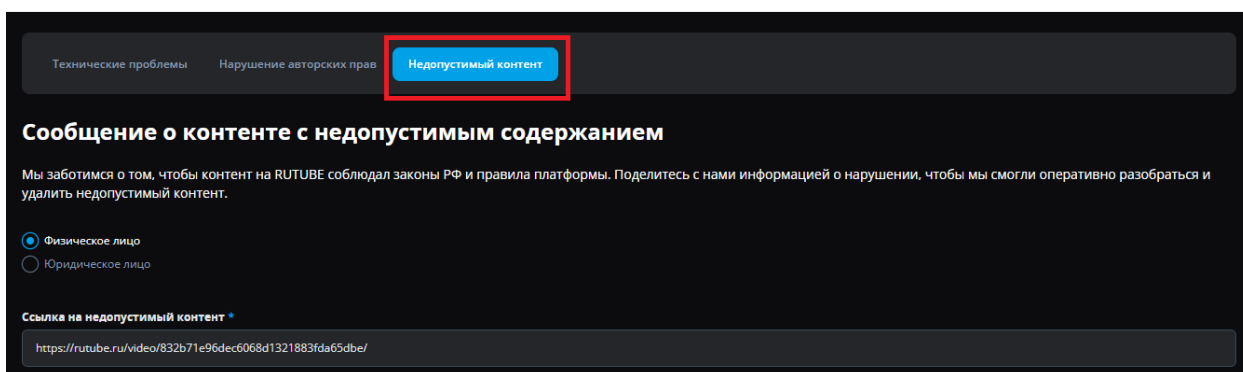


Рисунок 13 — Открытие формы жалобы на видео

Тест 7. Копирование ссылки на видео: Тест ищет «новости» и открывает первое видео (рис. 14). Затем нажимает кнопку «Поделиться» и выбирает

«Копировать ссылку» (рис. 15). После этого программа проверяет, что ссылка успешно сохранилась в буфер обмена компьютера.

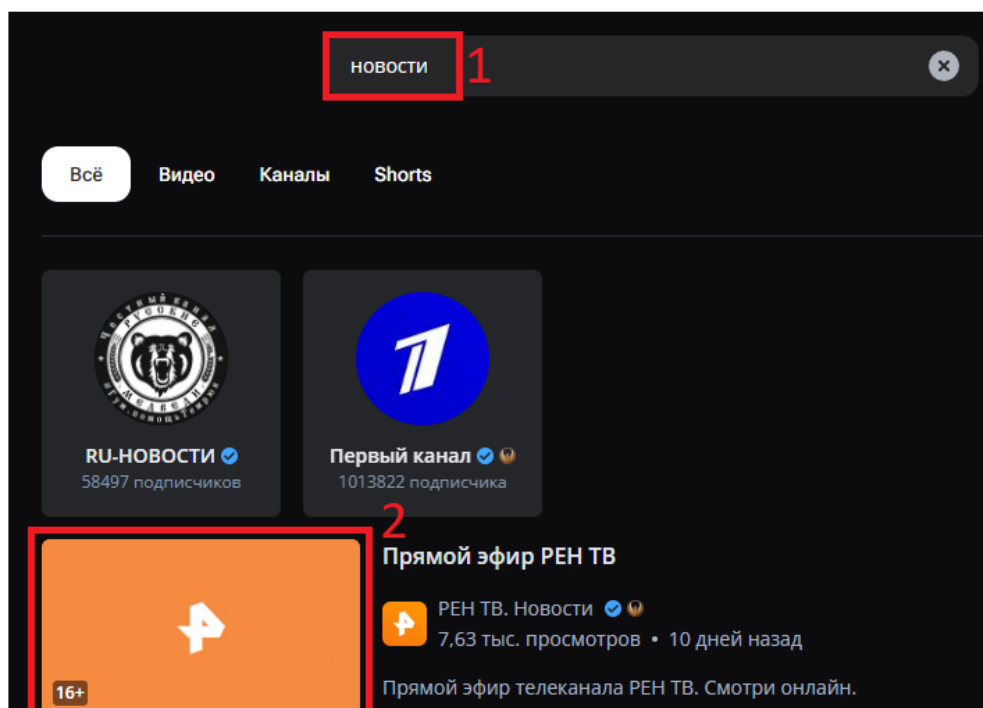


Рисунок 14 — Ввод поискового запроса «Новости» и открытие первого видео

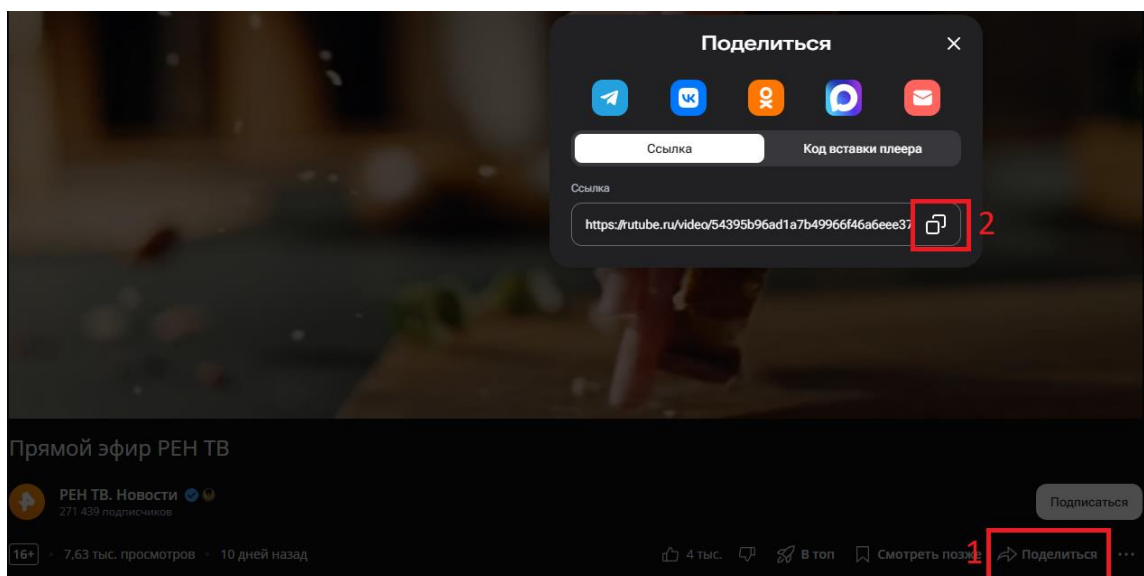


Рисунок 15 — Клик по кнопке «Поделиться» и «Копировать ссылку»

Тест 8. Добавление видео в «Смотреть позже»: Тест находит видео по запросу «музыка», открывает его контекстное меню и нажимает «Смотреть позже» (рис. 16). После этого переходит в раздел «Смотреть позже» в левом меню и проверяет, что ролик появился в плейлисте (рис. 17).

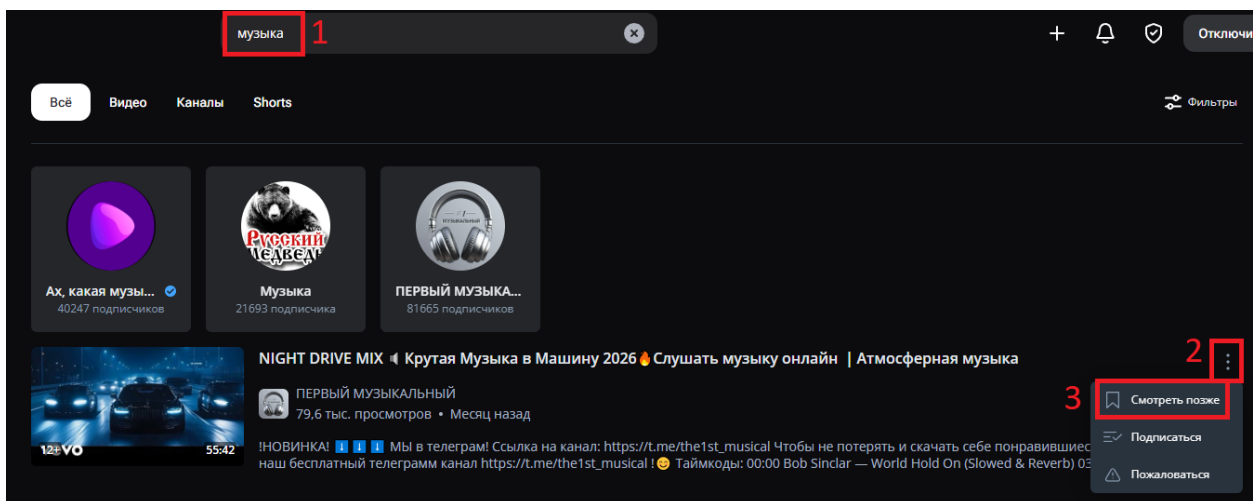


Рисунок 16 — Ввод поискового запроса «музыка», открытие меню и клик по кнопке «Смотреть позже»

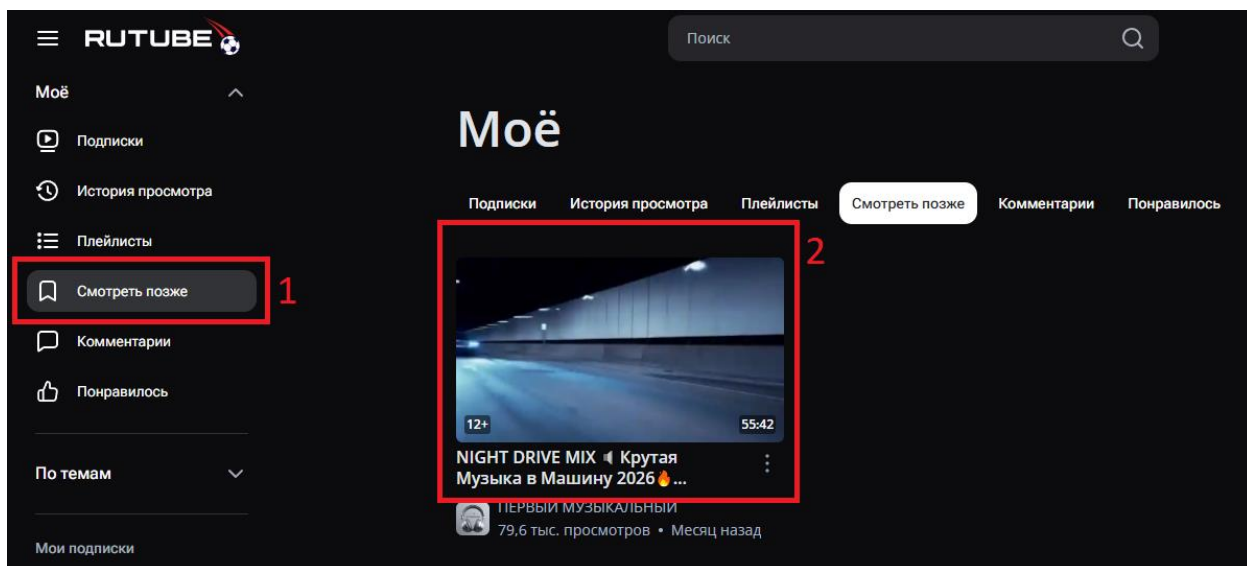


Рисунок 17 — Переход в раздел «Смотреть позже» и проверка появления ролика

Тест 9. Оценка видео (Лайк/Дизлайк): Тест открывает видео по запросу «музыка», нажимает кнопку «Лайк», а потом сразу же «Дизлайк» (рис. 18). Затем переходит в раздел «Понравилось» и проверяет, что этого видео там нет, так как система корректно учла последнюю оценку (рис. 19).

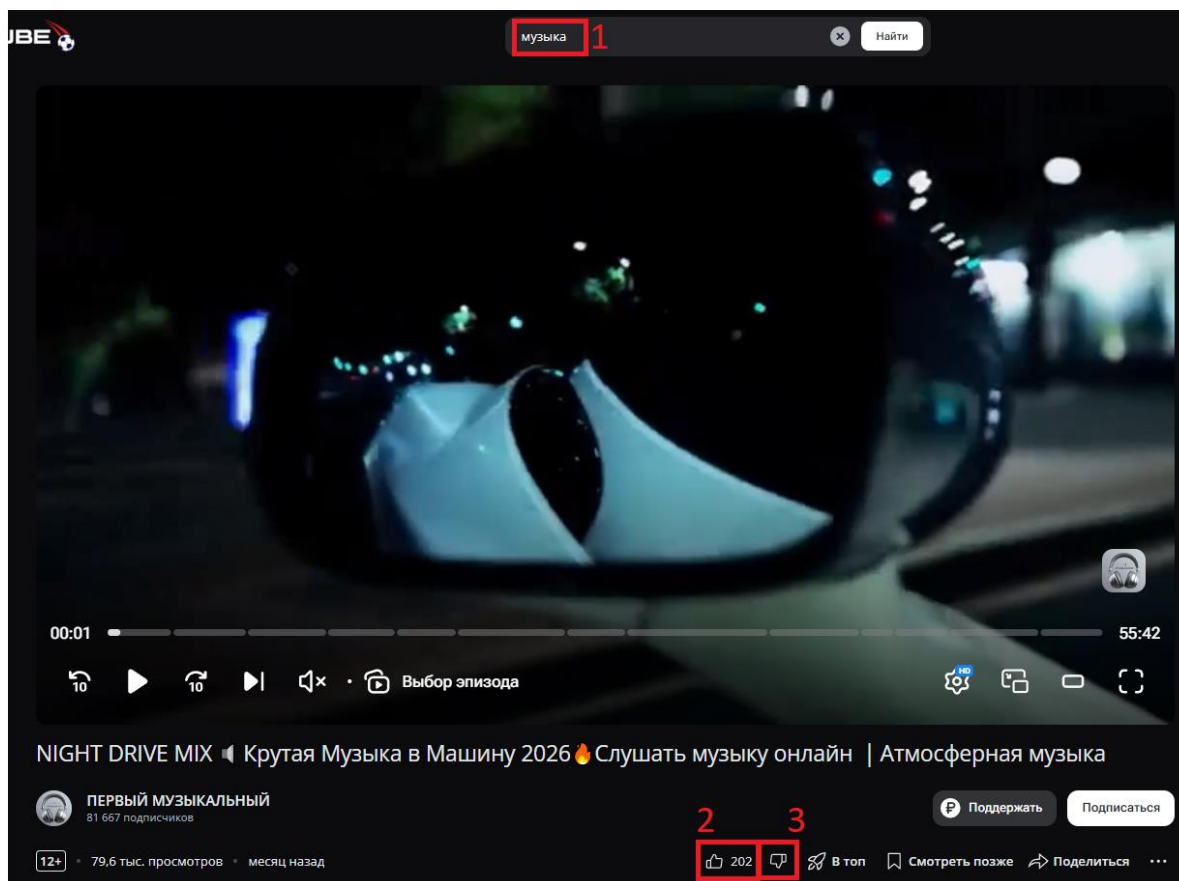


Рисунок 18 — Ввод поискового запроса «музыка», открытие видео, нажатие кнопки «Лайк» и «Дизлайк»

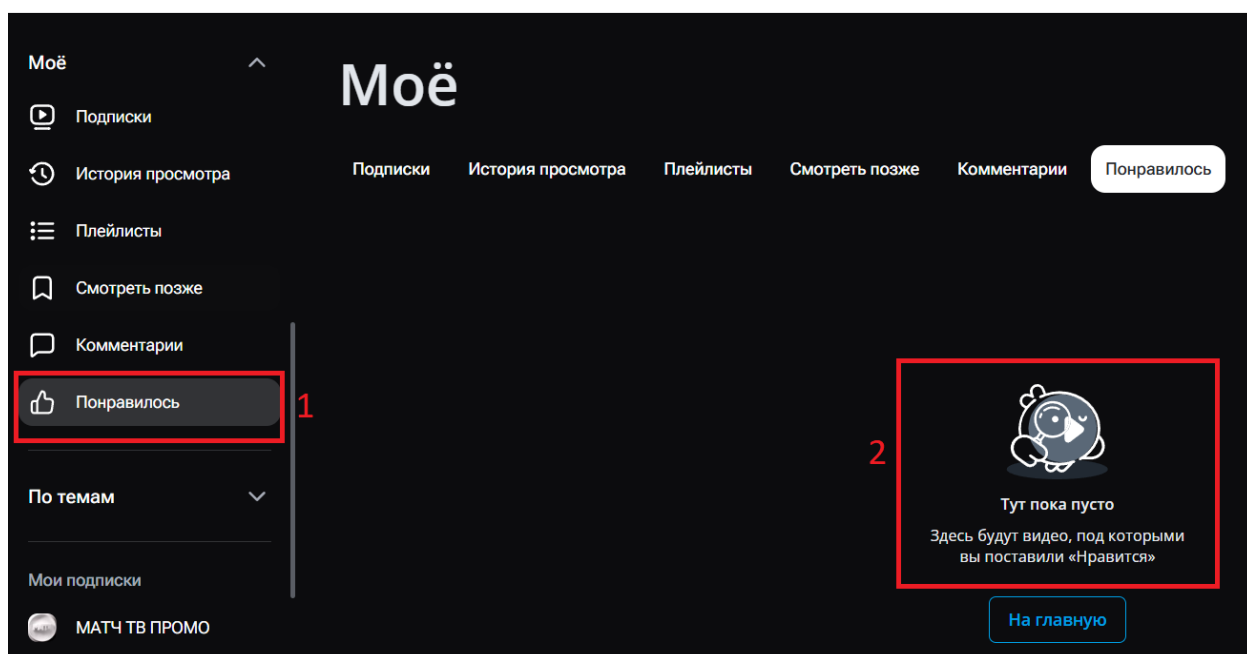


Рисунок 19 — Открытие раздела «Понравилось» и проверка отсутствия видео

Тест 10. Настройка видео: Тест открывает видео по запросу «музыка» и заходит в настройки плеера (шестеренка). Затем выбирает качество «480p» и нажимает кнопку «Полноэкранный режим» (рис. 20). В конце проверяется, что видео расширилось, а качество 480p успешно применилось.

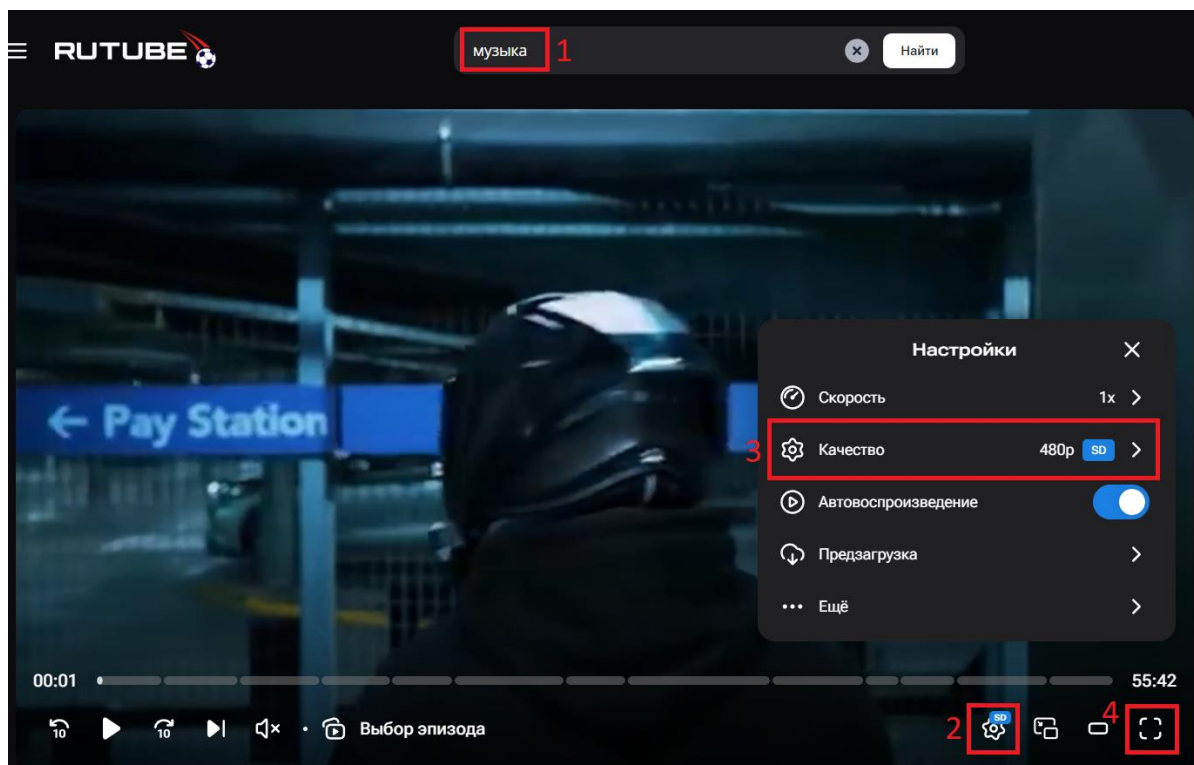


Рисунок 20 — Ввод поискового запроса «музыка», открытие видео, выбор качества «480p» и переход в «Полноэкранный режим»

2.1.4 Результат выполнения всех тестов

Все тесты были запущены вместе в среде разработки IntelliJ IDEA через JUnit 5. Для выполнения тестов, требующих авторизации профиля (подписки, лайки, жалобы, плейлисты), система перед каждым тестом автоматически загружала данные из файла cookies.json. Это позволило полностью пройти весь цикл проверок без ручного входа и ввода.

По итогу все тесты успешно прошли проверку и получили статус «Passed» (см. рис. 21). Программа не выявила зависаний элементов интерфейса или ошибок при загрузке страниц, что говорит о стабильной работе проверенных функций сайта и правильной настройке самого кода автотестов.

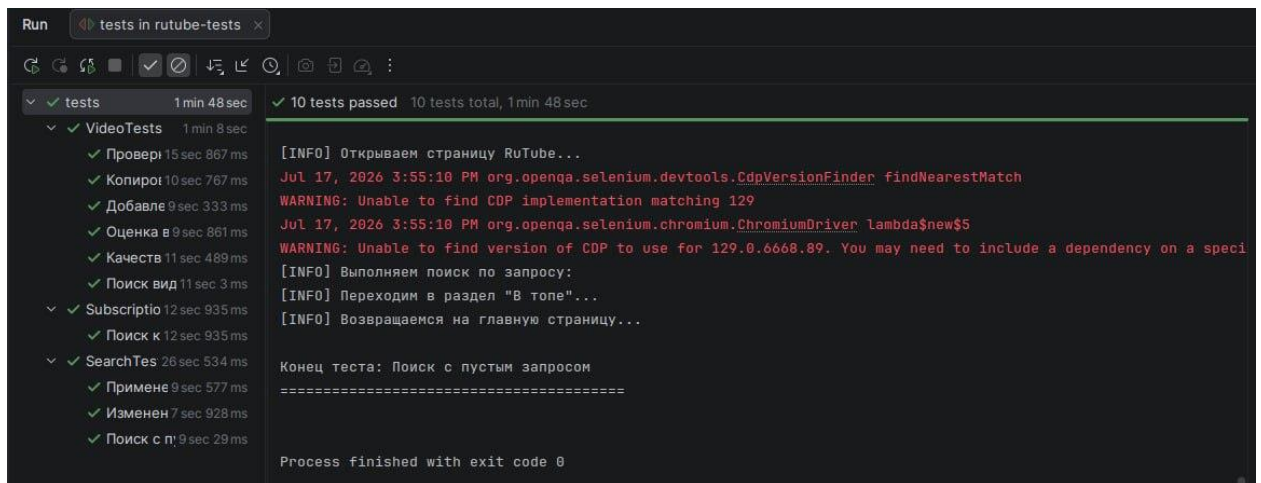


Рисунок 21 — Скриншот логирования

2.2 Описание классов и методов

Архитектура проекта построена на основе Page Object Model (POM). Для удобства и читаемости весь программный код разделен на функциональные блоки: базовые классы, элементы интерфейса, страницы, утилиты и классы с тестами.

2.2.1. Блок базовых классов (base)

В этом разделе хранятся родительские классы, которые задают общую логику для всего проекта.

- **BaseTest** — фундамент для всех тестовых классов. Содержит методы, которые выполняются до и после каждого теста:
 - `setUp(TestInfo testInfo)` — начальная настройка: задает размер окна, открывает стартовую страницу, закрывает всплывающие окна, подгружает куки для авторизации и делает перезагрузку страницы (`refresh`) для применения настроек. Также автоматически фиксирует название запущенного теста в консоли через логгер.
 - `closePopups()` — автоматический поиск и закрытие рекламных баннеров и окон с соглашением об использовании файлов куки.
 - `loadCookiesFromFile(String filePath)` — программная авторизация тестового профиля путем подгрузки сохраненной сессии из JSON-файла.

- `tearDown(TestInfo testInfo)` — корректное завершение работы веб-драйвера и вывод сообщения об окончании теста.
- `BasePage` — родительский класс для всех страниц. Содержит метод:
 - `isDisplayed()` — проверяет, успешно ли загрузилась требуемая страница на экране пользователя (ищет корневой элемент страницы).
- `BaseElement` — родительский класс для интерактивных элементов. Содержит общие методы:
 - `isDisplayed()` — проверяет, виден ли конкретный элемент на экране в данный момент, ожидая его появления до 10 секунд.
 - `waitForLoad()` — заставляет тест сделать паузу и гарантированно дожидаться, пока элемент загрузится на странице.
 - `getText()` — считывает и возвращает текстовое содержимое, которое написано внутри элемента.
 - `getBaseElement()` — технический метод, дающий прямой доступ к элементу для выполнения нестандартных команд библиотеки Selenide.

2.2.2 Блок элементов интерфейса (elements)

В этом разделе находятся классы-обертки для стандартных веб-элементов сайта. Они наследуют общую логику от `BaseElement` и реализуют функции конкретных элементов веб-страницы.

- `Button`, `Link`, `Tab` – классы для взаимодействия с кнопками, ссылками и вкладками. Содержат методы:
 - `click()` — имитирует стандартное нажатие по выбранному элементу.
 - `byText()`, `byClass()`, `byXpath()`, `byHref()` — статические методы поиска, которые помогают оперативно найти нужный элемент

на странице по его тексту, классу или адресу ссылки, избавляя от необходимости прописывать длинные пути.

- **Input** – класс для работы с текстовыми полями ввода. Содержит методы:
 - `fill(String value)` — умный ввод текста: метод сначала очищает поле (если есть кнопка очистки), затем печатает нужный текст и нажимает клавишу Enter.
 - `clear()` — выполняет стандартную очистку поля от напечатанных символов.
 - `withClearButton(Button button)` — связывает поле ввода с кнопкой-крестиком, чтобы можно было сразу сбрасывать старый текст в поиске.
 - `byId(String id)`, `byName(String name)`, `byPlaceholder(String placeholder)`, `byClass(String className)` — методы для поиска нужного поля ввода по его параметрам в HTML-коде: по идентификатору, имени, тексту-подсказке или классу.
- **Dropdown** – класс для взаимодействия с выпадающими списками. Содержит методы:
 - `selectOption(String optionText)` — кликает по выпадающему списку для его раскрытия, находит в нем вариант с нужным текстом и выбирает его.
 - `byClass(String className)` — метод для поиска нужного выпадающего списка на странице по имени его класса.
- **VideoPlayer** – класс для управления встроенным медиаплеером сайта. Содержит методы:
 - `clickWithJS(SelenideElement element)` и `showElement(SelenideElement element)` — скрытые технические функции, использующие JavaScript для взаимодействия с кнопками плеера. Это необходимо для обхода

проблемы, когда элементы перекрыты другими объектами на странице.

- `getPlayer()` — возвращает готовый объект плеера для начала работы с ним.
- `pause()` — наводит мышь на плеер и ставит видео на паузу, если оно воспроизводится.
- `isPaused()` — проверяет, находится ли видео на паузе в данный момент.
- `openSettingsMenu()` — технический метод, который открывает панель настроек внутри плеера.
- `setQuality(String quality)` — открывает меню настроек плеера и выбирает нужное разрешение видео.
- `getCurrentQuality()` — считывает информацию из настроек плеера и возвращает текст с текущим качеством воспроизведения.
- `toggleFullscreen()` — наводит курсор на плеер и нажимает кнопку разворачивания видео на весь экран.
- `waitForLoad()` — заставляет тест дожидаться полной загрузки плеера, прежде чем выполнять с ним какие-либо действия.
- `hover()` — имитирует движение мыши по плееру, чтобы на экране появились скрытые элементы управления (кнопка паузы, шестеренка настроек и т.д.).

2.2.3 Блок страниц (pages)

В этом разделе находятся классы, которые описывают конкретные веб-страницы сайта. Каждый класс содержит методы для выполнения действий, доступных пользователю на соответствующей странице.

- `VideoPage` – класс для работы со страницей просмотра видеоролика.

Содержит методы:

- `switchToNewWindow()` — метод, переключающий внимание автотеста на новую открытую вкладку браузера.

- `getVideoPlayer()` — предоставляет доступ к объекту плеера для прямого управления воспроизведением.
- `getCurrentQuality()`, `isPaused()`, `setQuality(String quality)`, `pause()`, `toggleFullscreen()` — методы управления плеером, которые передают команды напрямую в плеер, не перегружая код самой страницы.
- `openMenu()` — нажимает на кнопку с тремя точками под видео для открытия контекстного меню.
- `addToWatchLater()` — открывает меню и нажимает на кнопку добавления видео в список «Смотреть позже».
- `copyLink()` — нажимает кнопку «Поделиться», а затем выбирает опцию копирования ссылки на ролик.
- `isLinkCopied()` — проверяет, успешно ли сохранилась ссылка на сайт rutube.ru в системном буфере обмена устройства.
- `like()`, `dislike()` — нажимают соответствующие кнопки оценки видеоролика.
- `reportVideo()` — открывает меню, нажимает кнопку «Пожаловаться» и переключается на открывшуюся вкладку с формой.
- `isComplaintFormDisplayed()` — проверяет, успешно ли загрузилась страница с формой для отправки жалобы на контент.
- `closeComplaintForm()` — закрывает текущую вкладку с формой жалобы и возвращает управление на основную вкладку с видео.
- `getVideoTitle()` — считывает и возвращает текстовый заголовок текущего видеоролика.
- `isFullscreen()` — проверяет по атрибутам кнопки разворачивания, открыто ли видео на весь экран в данный момент.

- **ChannelPage** – класс для работы со страницей канала пользователя.
Содержит методы:
 - `subscribe()` — проверяет наличие кнопки подписки, и если еще нет подписки, нажимает на нее
 - `isSubscribed()` — проверяет, изменился ли текст на кнопке подписки на статус «Вы подписаны».
- **MainPage** – класс, представляющий стартовую страницу сайта.
Содержит методы:
 - `search(String query)` — вводит переданный текст в поисковую строку (или оставляет ее пустой, если текста нет), нажимает клавишу поиска и переходит на страницу результатов.
 - `goToTop()` — открывает раздел популярных видеороликов «В топе» по прямой ссылке.
 - `openMainPage()` — возвращает пользователя на главную страницу кликом по логотипу или прямым переходом по ссылке.
- **PlaylistPage** – класс для взаимодействия со списками воспроизведения (плейлистами). Содержит методы:
 - `likedPlaylist()`, `watchLaterPlaylist()` — статические методы, которые открывают страницы плейлистов «Понравилось» или «Смотреть позже» соответственно.
 - `isVideoInPlaylist(String videoTitle)` — ищет на странице плейлиста карточку видеоролика с заданным названием и возвращает успешный результат, если она найдена.
 - `isVideoNotInPlaylist(String videoTitle)` — убеждается, что видеоролика с таким названием нет в текущем списке.
- **SearchPage** – класс для работы со страницей результатов поиска.
Содержит методы:

- `openFilters()` — дожидается загрузки результатов, проверяет, скрыта ли панель фильтров, и если скрыта — нажимает кнопку «Фильтры» для ее отображения.
- `applyFilter(String filterValue)` — находит на панели фильтр с нужным названием, нажимает на него и дожидается его активации.
- `openFirstVideo()` — кликает по первому видеоролику в списке результатов для перехода к его просмотру.
- `getFirstVideoTitle()`, `getFirstVideoPublishDate()`, `getFirstVideoDuration()` — считывают с карточки первого видеоролика его название, дату публикации и длительность соответственно.
- `getSearchInputValue()` — возвращает текст, который в данный момент введен в поисковую строку.
- `goToChannelsTab()` — переключает результаты поиска на вкладку с найденными каналами.
- `subscribeToFirstChannelAndGoToChannel()` — нажимает кнопку подписки на первом канале в списке, а затем переходит на его страницу.
- `clearSearch()` — нажимает на крестик в строке поиска для удаления введенного текста.
- `searchAgain(String query)` — очищает строку поиска, вводит новый запрос и запускает его.
- `isFiltersOpened()`, `waitForResultsLoad()`, `findFilterOption()`, `waitForFilterActivation()`, `getFirstVideoAttribute()` — скрытые технические методы для проверки состояния элементов и поиска информации внутри карточек видео.

2.2.4 Блок тестов (tests)

В этом разделе хранятся сами сценарии автоматизированных проверок, разделенные по смыслу на несколько классов. Также внутри каждого теста есть пошаговое логирование действий пользователя

- SearchTests – набор проверок поисковой системы. Содержит сценарии:
 - applyFilters() — ищет новости, применяет фильтры по дате и длительности, затем проверяет, что первое видео в списке действительно соответствует этим строгим критериям.
 - changeSearchQuery() — делает запрос, стирает его, вводит новый и убеждается, что текст в строке и результаты выдачи успешно обновились.
 - emptySearch() — пытается выполнить поиск без ввода текста, переходит в другой раздел и возвращается обратно, чтобы проверить, что сайт не выдает ошибку и продолжает работать корректно.
- SubscriptionTests – набор проверок для работы с подписками. Содержит сценарии:
 - searchChannelAndSubscribe() — находит через поиск заданный канал, оформляет на него подписку из списка результатов и проверяет на странице канала, что подписка оформлена.
- VideoTests Набор – проверок для работы с плеером и оценки видеороликов. Содержит сценарии:
 - searchVideoAndPause() — находит видео, включает его и ставит на паузу, проверяя, что воспроизведение действительно остановилось.
 - sendComplaint() — открывает меню видеоролика, вызывает форму отправки жалобы на контент и закрывает ее без ошибок.
 - copyVideoLink() — нажимает кнопку «Поделиться» и проверяет, что ссылка скопировалась в системный буфер обмена.

- `addToWatchLater()` — сохраняет ролик в плейлист для отложенного просмотра и проверяет его фактическое появление в этом плейлисте.
- `likeAndDislike()` — ставит ролику лайк, затем меняет оценку на дизлайк и убеждается, что видео пропало из списка понравившихся.
- `videoSettings()` — запускает ролик, выставляет конкретное качество изображения и переводит плеер в полноэкранный режим работы.

2.2.5 Блок утилит (utils)

В этом разделе находятся вспомогательные классы с инструментами общего назначения.

- `Utils` – класс со статическими методами. Содержит методы:
 - `parseDuration(String duration)` — принимает текстовую строку с длительностью видео (например, в формате минут и секунд) и математически переводит ее в общее количество секунд для удобства проведения проверок в тестах.
- `Logger` — класс для ведения журнала событий (логирования) в консоли. Содержит методы:
 - `info(String message)` — выводит в консоль текстовое сообщение о том, какое действие сейчас выполняет автотест.
 - `testStart(String testName)` — визуально отделяет начало нового теста в консоли с помощью разделительной линии и выводит его название.
 - `testEnd(String testName)` — фиксирует завершение теста и также выводит разделительную линию для удобства чтения сообщений.

2.2.6 Диаграмма классов (UML)

Для наглядного представления структуры разработанного проекта автоматизации тестирования и демонстрации связей между его компонентами была построена общая UML-диаграмма (рис. 25), а также классов элементов (рис. 22) и страниц (рис. 23). Данная схема отражает иерархию наследования, а также взаимодействие базовых классов, элементов интерфейса, страниц веб-ресурса, тестовых сценариев и утилит (рис. 24). Использование объектно-ориентированного подхода и шаблона Page Object Model (POM) позволило изолировать логику работы с веб-элементами от самих автотестов.

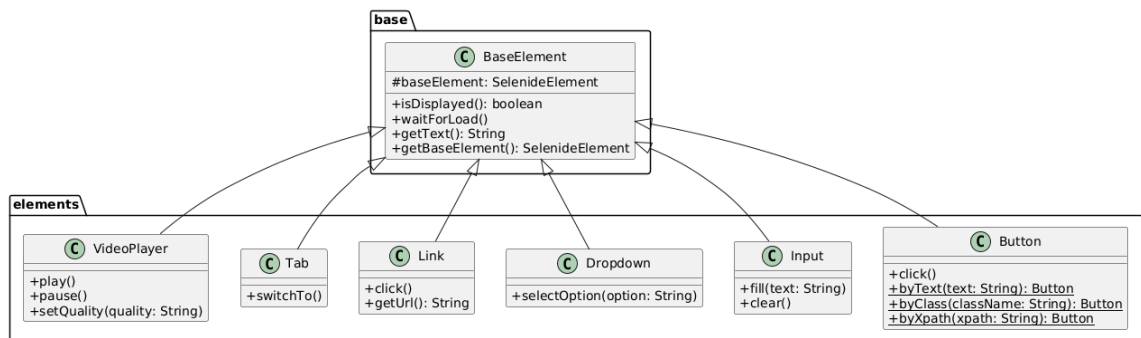


Рисунок 22 — Диаграмма классов элементов (Пакет elements)

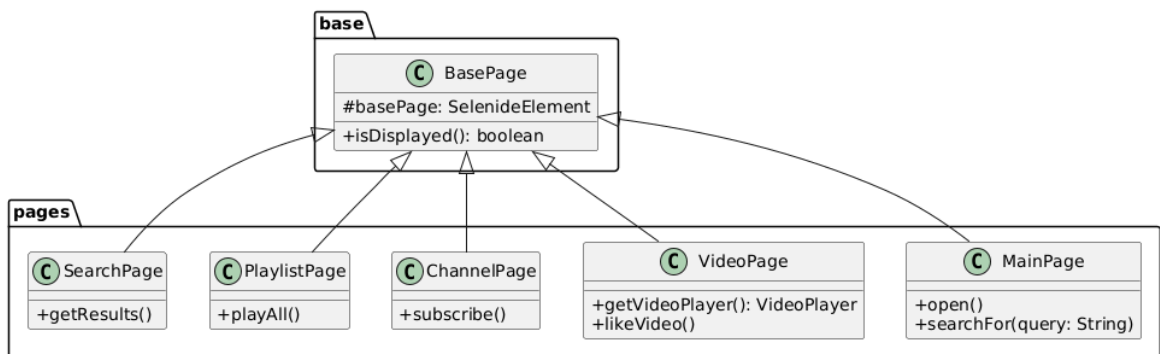


Рисунок 23 — Диаграмма классов страниц (Пакет pages)

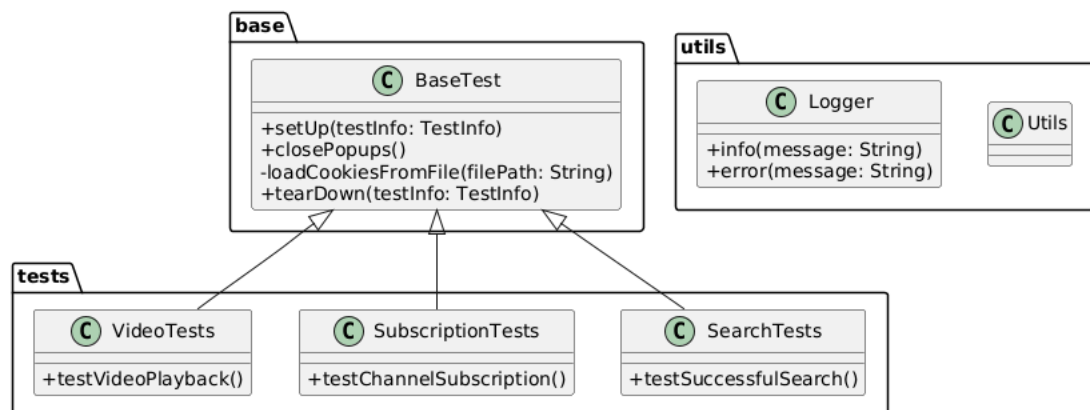


Рисунок 24 — Диаграмма классов тестов и утилит (tests и utils)

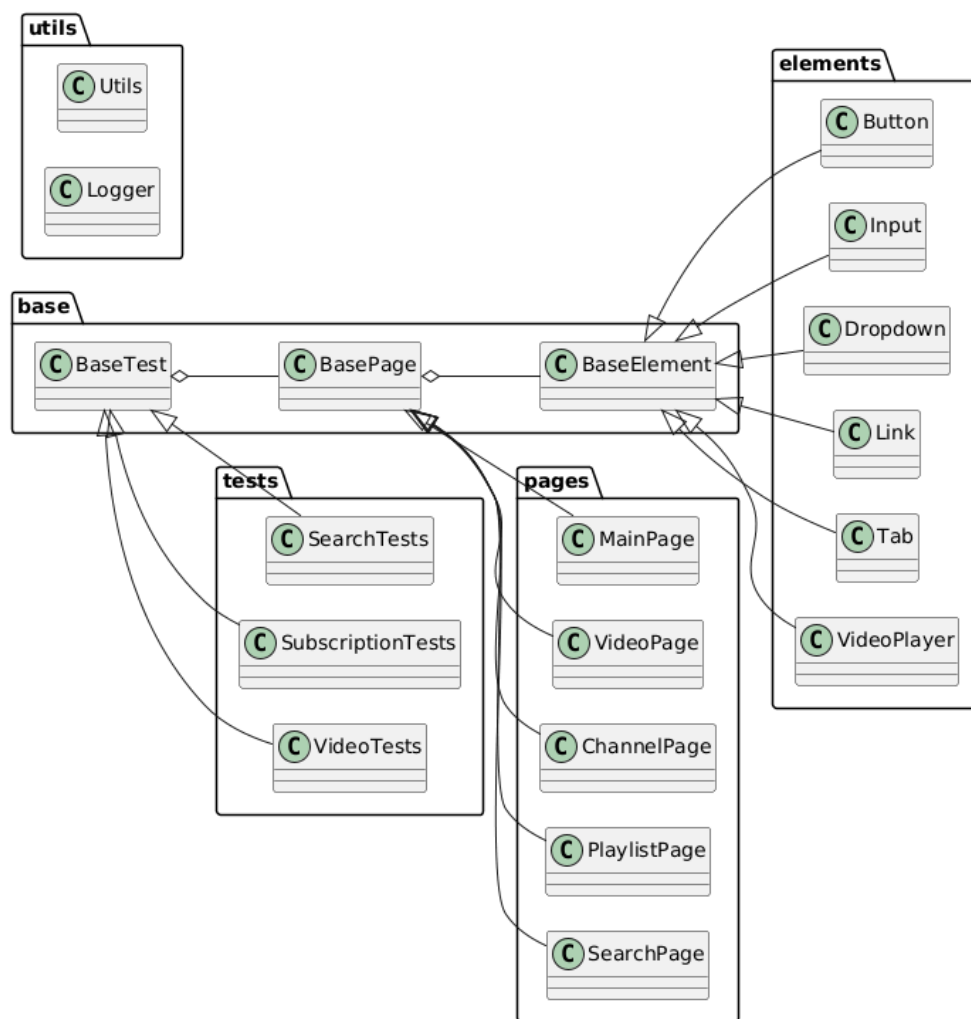


Рисунок 25 — Общая UML-диаграмма

3 ИНДИВИДУАЛЬНАЯ РАБОТА

В рамках учебной практики мной была выполнена работа по созданию базовой архитектуры тестового проекта, настройке среды разработки и приведению кода в соответствие с требованиями руководителя. Ниже представлено подробное описание выполненной работы.

3.1 Создание структуры проекта

Первоначальной задачей являлась организация правильной структуры тестового проекта, отвечающей принципам Page Object Model (POM). Мной была разработана иерархия пакетов, обеспечивающая логическое разделение компонентов проекта:

- Пакет `base` — для хранения базовых классов, содержащих общую логику для всех элементов, страниц и тестов;
- Пакет `elements` — для классов-обёрток UI-элементов (кнопки, поля ввода, ссылки и т.д.);
- Пакет `pages` — для Page Object классов, соответствующих страницам веб-приложения;
- Пакет `tests` — для размещения непосредственно тестовых классов;
- Пакет `utils` — для вспомогательных утилит.

Такая организация позволила обеспечить высокую поддерживаемость кода: каждый компонент находится в своём логическом месте, а разработчики могут легко находить и изменять необходимые части проекта.

3.2 Настройка файла сборки `pom.xml`

Мной был создан и настроен файл сборки проекта `pom.xml` с использованием системы управления зависимостями Maven. В файл были добавлены все необходимые зависимости:

- Selenium (версия 6.19.1) — фреймворк для UI-тестирования, обеспечивающий удобное взаимодействие с браузером;
- JUnit Jupiter (версия 5.10.0) — фреймворк для написания и выполнения тестов;

- AssertJ Core (версия 3.24.2) — библиотека для написания выразительных проверок (assertions);
- SLF4J Simple (версия 2.0.9) — библиотека для логирования;
- WebDriverManager [7] (версия 5.6.2) — инструмент для автоматического управления веб-драйверами.

Кроме того, был настроен плагин Maven Surefire для корректного запуска тестов, а также указаны версии компилятора Java 17 [8].

3.3 Создание иерархии базовых классов

Для обеспечения единообразия кода и исключения дублирования функциональности мной были реализованы базовые классы:

- BaseElement — абстрактный базовый класс для всех UI-элементов. Инкапсулирует общую логику поиска элементов по XPath, ожидание загрузки и проверку отображения. Все классы-обёртки (Button, Input, Link и др.) наследуются от него.
- BasePage — абстрактный базовый класс для всех Page Object'ов. Хранит корневой элемент страницы и предоставляет метод для проверки её отображения.
- BaseTest — базовый класс для всех тестов. Содержит методы настройки окружения: открытие браузера с заданными параметрами, загрузку сессионных cookies для авторизации, закрытие всплывающих окон, а также корректное завершение сессии браузера после каждого теста.

3.4 Работа над документацией

В соответствии с замечаниями руководителя практики мной была проведена работа по документированию кода:

- Добавлена Javadoc-документация для всех классов, описывающая их назначение и основные методы. Каждый класс получил краткое описание своего функционального назначения.

- Удалена вся лишняя документация: встроенные комментарии, начинающиеся с `//`, которые описывали очевидные действия в методах или перед локальными переменными. Также были удалены излишние описания конструкторов, не несущие смысловой нагрузки.
- Для публичных методов была добавлена документация с указанием параметров и возвращаемых значений, что упрощает понимание кода другими разработчиками.

3.5 Сортировка методов в классах

Мной была проведена работа по упорядочиванию методов во всех классах проекта в соответствии с установленным правилом: сначала `public` методы, затем `protected` и `private`, и наконец `static`. Это позволило добиться единообразия структуры классов: разработчику теперь легче ориентироваться в коде, так как наиболее важные (публичные) методы всегда находятся в начале класса.

3.6 Результаты выполненной работы

В результате проделанной работы была создана полноценная архитектура тестового проекта, готовая к дальнейшей разработке. Мой вклад обеспечил:

- Чёткую и логичную структуру проекта, удобную для командной работы;
- Корректно настроенную среду сборки с актуальными зависимостями;
- Единообразие стиля кода и документации во всех классах;

Данная основа была использована всеми членами команды для написания конкретных тестовых сценариев, описанных в предыдущих разделах отчёта.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В процессе выполнения учебной практики мной был использован следующий стек технологий, инструментов и библиотек.

4.1 Язык программирования

Java 17 — основной язык программирования, на котором написан весь тестовый проект. Выбор версии 17 обусловлен тем, что это долгосрочно поддерживаемая (LTS) версия, предоставляющая современные возможности языка. Java использовалась для написания всех классов проекта: базовых классов, классов-обёрток элементов, Page Object'ов, тестов и утилит.

4.2 Система сборки

Apache Maven — система автоматической сборки проектов, используемая для управления зависимостями, компиляции кода и запуска тестов. Основные задачи, решённые с помощью Maven: подключение всех необходимых библиотек через центральный репозиторий, управление версиями зависимостей, компиляция кода и запуск тестов через плагин Surefire.

4.3 Фреймворки для тестирования

Selenide (версия 6.19.1) — основной фреймворк для UI-тестирования, использованный для взаимодействия с веб-страницами. Selenide предоставляет удобный API для поиска элементов, выполнения действий и проверок, автоматически управляет ожиданиями. В проекте Selenide использовался для открытия браузера, навигации по страницам, поиска элементов и выполнения пользовательских действий.

JUnit Jupiter (версия 5.10.0) — фреймворк для написания и выполнения модульных тестов. Использовался для обозначения тестовых методов с помощью аннотации `@Test`, настройки окружения перед каждым тестом с помощью `@BeforeEach` и очистки ресурсов после каждого теста с помощью `@AfterEach`.

AssertJ Core (версия 3.24.2) — библиотека для написания выразительных и читаемых проверок. В проекте использовалась для проверки

результатов тестов: сравнения фактического и ожидаемого состояний, проверки наличия или отсутствия элементов на странице, валидации числовых значений.

4.4 Инструменты для управления браузером

WebDriverManager (версия 5.6.2) — библиотека для автоматического управления веб-драйверами браузеров. Автоматически скачивает необходимую версию драйвера для используемого браузера. В проекте использовался в связке с Selenide для автоматического управления драйвером Chrome.

Google Chrome — основной браузер, используемый для выполнения тестов. Браузер открывался в полноэкранном режиме с разрешением 1920×1080.

4.5 Вспомогательные библиотеки

SLF4J Simple (версия 2.0.9) — библиотека для логирования, использовалась для вывода информационных сообщений в консоль.

Jackson ObjectMapper — библиотека для работы с форматом JSON, использованная для чтения файла с сессионными cookies. Обеспечивает автоматическую авторизацию на сайте RuTube.

4.6 Инструменты разработки

Git — система контроля версий, использованная для отслеживания изменений и совместной работы в команде.

GitHub — платформа для хостинга Git-репозиторий, использованная как центральное хранилище кода.

IntelliJ IDEA — среда разработки, использованная для написания кода, сборки и запуска тестов.

EditThisCookie — расширение для браузера Chrome, использованное для экспорта сессионных cookies авторизованной сессии на RuTube.

4.7 Используемые подходы и паттерны

Page Object Model (POM) — основной паттерн проектирования, применённый в архитектуре проекта. POM предполагает создание отдельных

классов для каждой страницы веб-приложения, инкапсулирующих локаторы элементов и методы взаимодействия. Это позволяет отделить логику проверок от логики работы с интерфейсом и упрощает поддержку тестов.

Стандарт оформления кода — все классы проекта оформлены с использованием Javadoc-документации, методы отсортированы по уровням доступа, все строковые литералы и XPath-выражения вынесены в константы.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на отлично.
Отзыв руководителя с оценкой (см. рис. 26).

ОТЗЫВ

о прохождении учебной практики

Донцова Ирина Евгеньевна, студентка группы 4388 второго курса бакалавриата Санкт-Петербургского электротехнического университета им В.И. Ульянова (Ленина) “ЛЭТИ”, проходила учебную практику по теме “UI-тестирование” с 25.06.2025 по 08.07.2025 на кафедре МО ЭВМ.

Во время практики **Донцова Ирина Евгеньевна** выполнила все поставленные задачи, продемонстрировала владение языком программирования JAVA, умение планировать разработку и работать в команде.

По результатам учебной практики **Донцова Ирина Евгеньевна** заслуживает оценки **отлично**.

Подпись руководителя практики:

Ассистент кафедры
МОЭВМ



Шевелева А.М.

17.07.2026

Рисунок 26 — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики была выполнена работа по разработке набора автоматизированных UI-тестов для веб-приложения RuTube. Все поставленные задачи были успешно решены в полном объёме.

Результаты по каждой задаче

Задача 1. Изучение автоматизации с использованием языка Java и библиотеки Selenide. В процессе работы были освоены основы автоматизации тестирования пользовательского интерфейса с использованием языка Java и библиотеки Selenide. Изучены принципы работы с веб-драйверами, методы поиска элементов на странице, работа с ожиданиями и управление браузером. Selenide показал себя как удобный и эффективный инструмент благодаря лаконичному API и автоматическому управлению ожиданиями.

Задача 2. Проектирование структуры проекта по принципу Page Object Model. Спроектирована и реализована иерархическая структура проекта на основе паттерна Page Object Model. Созданы базовые классы BaseElement, BasePage и BaseTest, от которых наследуются все элементы, страницы и тесты. Разработана пакетная структура, включающая пакеты base, elements, pages, tests и utils. Это позволило избежать дублирования кода и упростить добавление новых тестов.

Задача 3. Программное описание основных страниц и компонентов сайта. Разработаны Page Object классы для всех ключевых страниц: MainPage, SearchPage, VideoPage, ChannelPage и PlaylistPage. Для каждой страницы определены локаторы элементов и методы взаимодействия. Также созданы классы-обёртки для UI-элементов: Button, Input, Link, Dropdown, Tab и VideoPlayer.

Задача 4. Настройка автоматической авторизации через JSON-файлы с куками. Реализован механизм автоматической авторизации через загрузку сессионных cookies из JSON-файла с использованием библиотеки

Jackson ObjectMapper. Это позволило выполнять тесты, требующие авторизации, без ручного ввода логина и пароля.

Задача 5. Написание 10 автотестов. Разработано 10 автоматизированных тестов, покрывающих ключевые пользовательские сценарии: поиск и фильтрацию видео, управление воспроизведением, настройку качества, оценку контента, отправку жалобы, копирование ссылки, добавление в плейлист «Смотреть позже» и подписку на каналы. Тесты структурированы по трём классам: SearchTests, VideoTests и SubscriptionTests.

Общие итоги работы

Все 10 разработанных тестов успешно выполняются.

В ходе выполнения тестов выявлены особенности платформы: перекрытие элементов управления плеера (использованы клики через JavaScript), задержка при применении фильтров (увеличены таймауты ожидания). Для обхода двухфакторной аутентификации реализован механизм загрузки сессионных cookies.

Выводы

Цель учебной практики — приобретение практических навыков разработки автоматизированных UI-тестов — полностью достигнута. В процессе работы освоены: работа с Maven, написание тестов с использованием Selenide и JUnit 5, применение паттерна Page Object Model, использование AssertJ для проверок, работа с cookies, документирование кода и командная разработка с Git и GitHub.

Разработанный проект может быть использован для дальнейшего расширения тестового покрытия и интеграции в процессы CI/CD. Полученный опыт может быть применён при работе над другими веб-проектами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Selenide: лаконичный фреймворк для UI-тестирования на Java [Электронный ресурс]. URL: <https://selenide.org/> (дата обращения: 10.07.2026).
2. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 10.07.2026).
3. AssertJ — Fluent assertions for Java [Электронный ресурс]. URL: <https://assertj.github.io/doc/> (дата обращения: 10.07.2026).
4. Apache Maven Documentation [Электронный ресурс]. URL: <https://maven.apache.org/guides/> (дата обращения: 11.07.2026).
5. Page Object Model — Selenium Documentation [Электронный ресурс]. URL: https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/ (дата обращения: 11.07.2026).
6. Репозиторий проекта summer_practice [Электронный ресурс] / onyanyam. – URL: https://github.com/onyanyam/summer_practice.
7. WebDriverManager — Automated driver management [Электронный ресурс]. URL: <https://github.com/bonigarcia/webdrivermanager> (дата обращения: 11.07.2026).
8. Java Documentation — Oracle [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/> (дата обращения: 11.07.2026).